

ME315A LAB03 RA223062

Laboratório 3: Dados relacionais e operações do tipo join

Questões

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.
- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;
- Para o arquivo de vôos:
 - a. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;
 - e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 - f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
col_flights <- cols_only(
  DESTINATION_AIRPORT = "c",
  ARRIVAL_DELAY = "d"
)

flights_function <- function(input, pos) {
  input %>%
    filter(
      !startsWith(DESTINATION_AIRPORT, "1"),
      !is.na(DESTINATION_AIRPORT),
      !is.na(ARRIVAL_DELAY)
    ) %>%
```

```

group_by(DESTINATION_AIRPORT) %>%
summarise(
  delay_sums = sum(ARRIVAL_DELAY),
  flights_total = n(),
  .groups = "drop"
)
}

flights <- read_csv_chunked(file = "flights.csv",
                           col_types = col_flights,
                           chunk_size = 1e6,
                           callback = DataFrameCallback$new(flights_function)
                           )

flights <- flights %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    delay_mean = sum(delay_sums) / sum(flights_total),
    .groups = "drop"
  )

col_airports <- cols_only(
  IATA_CODE = "c",
  CITY = "c",
  STATE = "c",
  LATITUDE = "d",
  LONGITUDE = "d"
)

airports <- read_csv("airports.csv", col_types = col_airports)

airlines <- read_csv("airlines.csv")

```

Rows: 14 Columns: 2

— Column specification —

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

2. Selecione a operação apropriada `join` para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

- Identifique a coluna que é a chave na tabela `flights`; Resposta: é a coluna "DESTINATION_AIRPORT".
- Identifique a coluna que é a chave na tabela `airports`; Resposta: é a coluna "IATA_CODE".
- Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?
Resposta: todos os aeroportos listados em `flights` estão presentes em `airports`.
- Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
- Armazene a tabela resultante no objeto `flights`.

```

flights <- flights %>%
  left_join(

```

```
airports %>% select(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE),
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
flights
```

A tibble: 322 × 6

	DESTINATION_AIRPORT	delay_mean	CITY	STATE	LATITUDE	LONGITUDE
	<chr>	<dbl>	<chr>	<chr>	<dbl>	<dbl>
1	ABE	5.80	Allentown	PA	40.7	-75.4
2	ABI	4.08	Abilene	TX	32.4	-99.7
3	ABQ	5.61	Albuquerque	NM	35.0	-107.
4	ABR	-3.39	Aberdeen	SD	45.4	-98.4
5	ABY	8.68	Albany	GA	31.5	-84.2
6	ACK	4.91	Nantucket	MA	41.3	-70.1
7	ACT	10.6	Waco	TX	31.6	-97.2
8	ACV	9.68	Arcata/Eureka	CA	41.0	-124.
9	ACY	11.8	Atlantic City	NJ	39.5	-74.6
10	ADK	-3.19	Adak	AK	51.9	-177.

i 312 more rows

```
missing_airports <- anti_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
missing_airports
```

A tibble: 0 × 6

i 6 variables: DESTINATION_AIRPORT <chr>, delay_mean <dbl>, CITY <chr>,

STATE <chr>, LATITUDE <dbl>, LONGITUDE <dbl>

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
table <- flights %>%
  group_by(STATE) %>%
  summarise(airports_by_state = n_distinct(DESTINATION_AIRPORT)) %>%
  arrange(desc(airports_by_state))
table
```

A tibble: 54 × 2

	STATE	airports_by_state
	<chr>	<int>
1	TX	24
2	CA	22
3	AK	19
4	FL	17
5	MI	15
6	NY	14
7	CO	10
8	MN	8
9	MT	8
10	NC	8

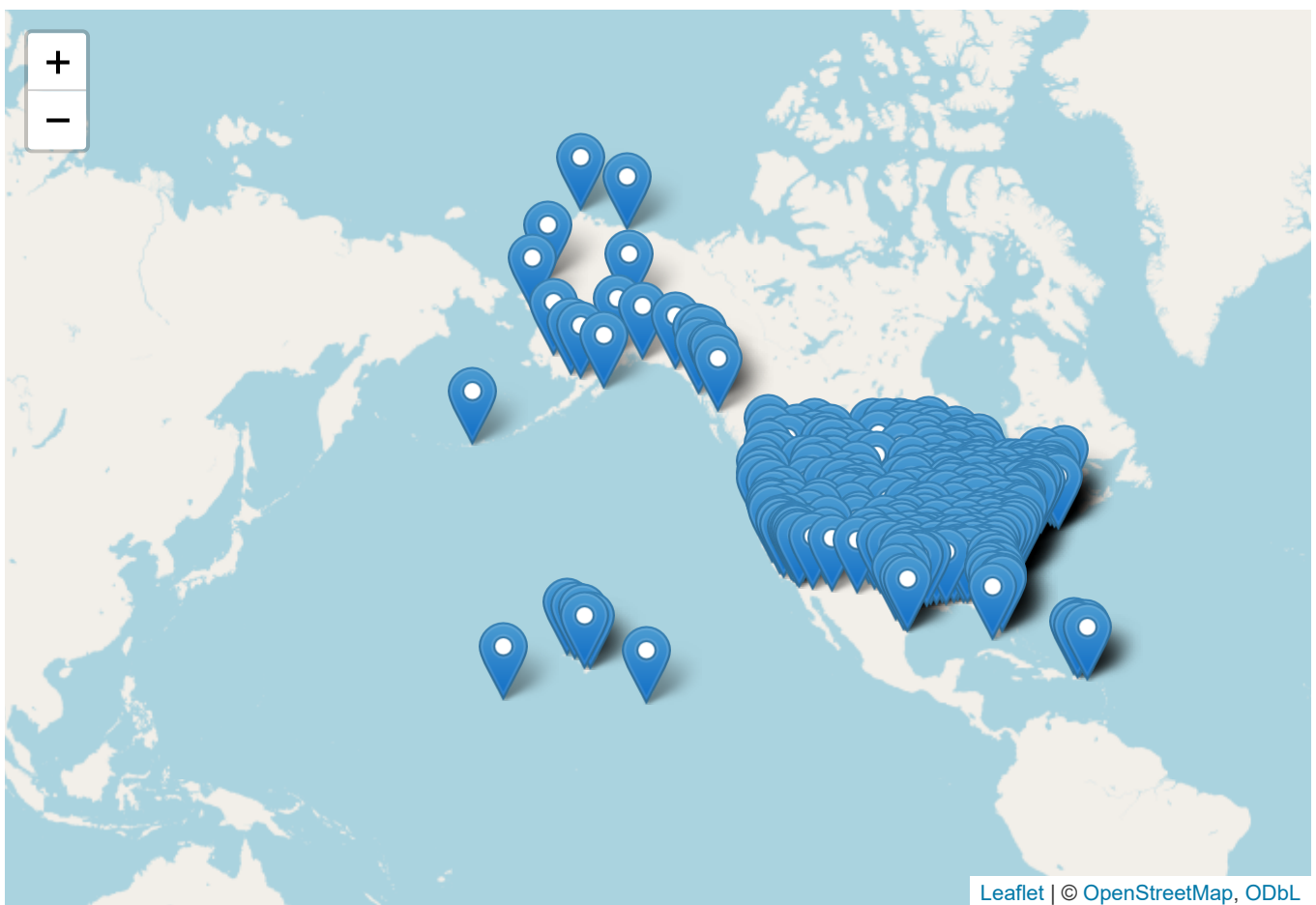
i 44 more rows

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

- Carregue o pacote `leaflet`;
- Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
- Armazene o grafico em b) numa variável chamada `this`.
- Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter): (NÃO FOI UTILIZADO JUPYTER, SOMENTE POSIT.CLOUD/RSTUDIO)

```
library(leaflet)

this <- leaflet(data = flights %>% filter(!is.na(LATITUDE), !is.na(LONGITUDE))) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE
  )
this
```



Horário de compilação do arquivo:

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-29 15:03:26 -03"
```